



验证对象: 同步 FIFO 的设计规范和代码

新手任务的验证对象

设计规范

模块名称

SyncFIFO

描述

此模块实现一个 32 位宽、容量为 16 个元素的同步 FIFO（先入先出）缓冲区。FIFO 用于暂存数据，提供写入和读取操作，并支持满/空状态指示。它具有时钟同步行为，适用于数据流处理、接口缓冲等场景。该设计遵循同步操作，使用单一时钟信号进行读写。

端口说明

端口名称	方向	宽度 (bit)	描述
clk	input	1	时钟信号
rst_n	input	1	低电平时初始化 FIFO

端口名称	方向	宽度 (bit)	描述
we_i	input	1	写使能信号，高有效， 当为 1 时，允许向 FIFO 写入数据。
re_i	input	1	读使能信号，高有效， 当为 1 时，允许从 FIFO 读取数据。
data_i	input	32	写入 FIFO 的数据
data_o	output	32	读取 FIFO 的数据
full_o	output	1	表明 FIFO 是否为满
empty_o	output	1	表明 FIFO 是否为空

功能描述

- 写入操作：
 - 当 `we_i` 为 1 时，FIFO 可以接收数据并存储到内部缓冲区 `ram` 中。
 - 写指针 `wptr` 指示下一个写入位置，随着每次写入操作递增。
 - 当 FIFO 已满时， `full_o` 为 1，写入无效。
- 读取操作：
 - 当 `re_i` 为 1 时，FIFO 将根据读取指针 `rptr` 从 `ram` 中输出数据。
 - 读取指针 `rptr` 指示下一个读取位置，随着每次读取操作递增。
 - 当 FIFO 为空时， `empty_o` 为 1，读取无效。
- 指针更新：
 - `wptr`（写指针）和 `rptr`（读指针）在时钟上升沿更新。 `rptr` 仅在 `re_i` 有效并且 FIFO 非空时更新， `wptr` 仅在 `we_i` 有效并且

FIFO 非满时更新。

- FIFO 操作时, 通过比较 `wptr` 和 `rptr` 的位置, FIFO 会自动调整数据的读写位置。

- **计数器:**

- `counter` 用于跟踪 FIFO 中的数据量 (从 0 到 16) 。每次写入数据时, `counter` 加 1, 每次读取数据时, `counter` 减 1。
- 当 `counter` 值为 0 时, `empty_o` 信号为 1, 表示 FIFO 为空; 当 `counter` 值为 16 时, `full_o` 信号为 1, 表示 FIFO 已满。

时序与复位

- **同步时序:** 所有的操作 (写入、读取、指针更新、计数器更新) 都在时钟信号的上升沿同步。
- **复位:** 在 `rst_n` 为低时, FIFO 内部所有指针 (`wptr` 、 `rptr`) 和数据输出 (`data_o`) 都将被清零, 并且计数器 `counter` 会被复位为 0。

功能块说明

1. 指针更新:

- 负责同步更新写入和读取指针。
- 在每个时钟周期内, 如果 `we_i` 为 1 并且 FIFO 不满, 则数据会被写入 FIFO, 且 `wptr` 自增。
- 如果 `re_i` 为 1 并且 FIFO 不空, 则会读取数据并将其输出, `rptr` 自增。

2. 计数器更新:

- 计数器 `counter` 用于追踪 FIFO 的当前数据量。
- 每次写入时, `counter` 自增; 每次读取时, `counter` 自减。

- FIFO 满时, counter 达到 16, full_o 为 1; FIFO 空时, counter 为 0, empty_o 为 1。

设计约束与假设

1. **数据宽度与深度**: 本设计采用 32 位宽度, 16 深度的 FIFO, 适用于较小规模的数据缓存需求。
2. **时钟域**: FIFO 模块假设在单一时钟域内工作, 且时钟信号与复位信号是同步的。
3. **数据保持**: FIFO 的数据存储在一个 16 个元素的 RAM 数组中, 每个元素为 32 位。
4. **读写并行**: FIFO 未满足不为空时, 允许同时进行读、写操作。

边界条件

1. 当 FIFO 已满且 we_i 信号为高时, 写入操作将被阻塞。
2. 当 FIFO 为空且 re_i 信号为高时, 读取操作将被阻塞。

设计代码

```
module SyncFIFO (  
    input wire    clk,  
    input wire    rst_n,  
    input wire    we_i,  
    input wire    re_i,  
    input wire [31:0] data_i,  
    output reg [31:0] data_o,  
    output wire    full_o,  
    output wire    empty_o  
);  
  
    reg [31:0] ram[16];  
  
    reg [3:0] wptr;  
    reg [3:0] rptr;  
    reg [4:0] counter;  
  
    wire rvalid, wvalid;  
  
    assign rvalid = re_i && !empty_o;  
    assign wvalid = we_i && !full_o;
```

```
always @(posedge clk) begin : PTR_UPDATE
    if (!rst_n) begin
        wptr <= 0;
        rptr <= 0;
        data_o <= 0;
    end else begin
        if (rvalid) begin
            rptr <= rptr + 1;
            data_o <= ram[rptr];
        end
        if (wvalid) begin
            wptr <= wptr + 1;
            ram[wptr] <= data_i;
        end
    end
end

always @(posedge clk) begin : COUNTER_UPDATE
    if (!rst_n) counter <= 0;
    else if (rvalid ^ wvalid) begin
        if (rvalid) counter <= counter - 1;
        if (wvalid) counter <= counter + 1;
    end
end

assign full_o = counter == 5'd16;
assign empty_o = counter == 0;

endmodule
```